Module – Conditional Statements: if, if–else, switch, and Modern Switch Expressions (→, yield)

The if Statement

Key Points
• Used for simple decision making.
• Evaluates a boolean expression — executes the block if true.
• Parentheses around the condition are mandatory.

Code Snippet

```
int marks = 85;
if (marks > 75) {
    System.out.println("Distinction");
}
```

Exercise 01 – IfCondition.java
Write a program that checks whether a number entered by the user is **positive, negative, or zero**, and prints an appropriate message.

The if–else Statement

Key Points

• Executes one block if condition is true, another if false.
• Only one branch executes at a time.
• Can be nested for multi-level decisions.

Code Snippet

```
int age = 20;
if (age >= 18) {
    System.out.println("Eligible to vote");
} else {
    System.out.println("Not eligible");
}
```

Exercise 02 – IfElse.java

Write a program that reads a number and prints **"Even"** if it's even, otherwise prints **"Odd"**.

Nested and Laddered Conditions

Key Points
• Nested if → condition inside another if.
• if–else if–else → used for multiple comparisons.
• Evaluated top to bottom; first true block executes.

Code Snippet

```
int marks = 82;
if (marks >= 90)
    System.out.println("Grade A");
else if (marks >= 75)
    System.out.println("Grade B");
else if (marks >= 60)
    System.out.println("Grade C");
else
    System.out.println("Fail");
```

Exercise 03 – IfElseLadder.java
Implement a **grading system** that assigns grades (A, B, C, D, F) based on percentage input.

Traditional switch Statement

Key Points

• Used for multiple value-based comparisons.
• Works with byte, short, char, int, String, and enums.
• Each case must end with `break` to prevent fall-through.

Code Snippet

```
String day = "MONDAY";
switch (day) {
    case "MONDAY":
        System.out.println("Start of week");
        break;
    case "FRIDAY":
        System.out.println("Weekend is near");
        break;
    default:
        System.out.println("Midweek day");
}
```

Exercise 04 – SwitchBasic.java

Write a switch statement that takes a **traffic light color** (“red”, “yellow”, “green”) and prints the **corresponding action** (“Stop”, “Wait”, “Go”).

Fall-Through Behaviour

Key Points

• If `break` is omitted, control "falls through" to the next case.
• Used intentionally for grouped logic.

Code Snippet

```
int day = 6;
switch (day) {
    case 6:
    case 7:
        System.out.println("Weekend");
        break;
    default:
        System.out.println("Weekday");
}
```

Exercise 05 – FallThrough.java

Write a switch that prints **"Weekend"** for both Saturday and Sunday numbers (6, 7) using intentional fall-through.

Enhanced switch (Java 14+)

Key Points
• Eliminates fall-through by design.
• Introduces arrow (→) syntax for concise code.
• Supports multiple labels per case.

Code Snippet

```
String day = "MONDAY";
switch (day) {
    case "MONDAY", "TUESDAY", "WEDNESDAY", "THURSDAY", "FRIDAY" ->
        System.out.println("Weekday");
    case "SATURDAY", "SUNDAY" ->
        System.out.println("Weekend");
}
```

Exercise 06 – EnhancedSwitch.java
Convert an **old-style switch** that prints weekday/weekend messages to the **arrow-style syntax**.

Switch as an Expression

Key Points

• New switch can return a value.
• Expression form ends with a result instead of `break`.
• Use `yield` for multi-statement blocks.

Code Snippet

```
String day = "MONDAY";
String type = switch (day) {
    case "SATURDAY", "SUNDAY" -> "Weekend";
    default -> "Weekday";
};
System.out.println(type);
```

Exercise 07 – SwitchExpression.java

Write a program that takes a **score range (A, B, C, D, F)** and returns **text feedback** (“Excellent”, “Good”, “Average”, “Poor”, “Fail”) using a switch expression.

Using yield for Multi-Line Cases

Key Points

• Use `yield` inside `{}` when a case has multiple statements.
• `yield` returns the result value from that case block.

Code Snippet

```
int marks = 85;
String grade = switch (marks / 10) {
    case 10, 9 -> "A";
    case 8 -> {
        System.out.println("Good effort!");
        yield "B";
    }
    case 7 -> "C";
    default -> "Fail";
};
System.out.println("Grade: " + grade);
```

Exercise 08 – SwitchYield.java

Modify the above program to **print encouragement messages** based on grades and use `yield` to return both message and grade concatenated.

Expression vs Statement Recap

Feature	Old Switch	Switch Expression
Syntax	`case:` + `break`	`case ->`
Fall-through	Allowed	Not allowed
Can return value	❌	✅
Multiple labels	Duplicated	Supported
Multi-line	With `break`	With `yield`

Exercise 09 – SwitchComparison.java
Rewrite a **nested if–else ladder** (for grade or day-type logic) using a **modern switch expression**, and compare line count and readability.

Key Takeaways

• if / if–else → for simple, range-based conditions.
• switch → efficient for fixed discrete values.
• Enhanced switch adds clarity, prevents fall-through, and supports returning values.
• Use → for concise single-line logic, yield for multi-line.
• Modern switch improves readability, safety, and is future-ready for pattern matching.

Module – Pattern Matching in switch (Java 21 Preview)

Introduction to Pattern Matching in Switch

Key Points

• Introduced as a preview feature in Java 21.
• Extends switch to handle object patterns directly.
• Allows type checks and casting to be done automatically.
• Eliminates need for explicit `instanceof` and casting blocks.

Code Snippet

```
static void printObject(Object obj) {
    switch (obj) {
        case Integer i -> System.out.println("Integer: " + i);
        case String s -> System.out.println("String: " + s);
        case null -> System.out.println("Null value");
        default -> System.out.println("Unknown type");
    }
}
```

Exercise 01 – SwitchPatternIntro.java

Write a method that accepts an `Object` and uses **pattern matching in switch** to print whether it's an `Integer`, `Double`, or `String`.

Traditional Approach vs Pattern Matching

Key Points

Aspect	Traditional Approach	Pattern Matching
Type check	Uses `instanceof`	Uses `case` patterns
Casting	Manual	Automatic
Readability	Verbose	Concise

Code Snippet

```
// Pre-Java 21
if (obj instanceof String) {
    String s = (String) obj;
    System.out.println(s.toUpperCase());
} else if (obj instanceof Integer) {
    Integer i = (Integer) obj;
    System.out.println(i + 10);
}

// Java 21
switch (obj) {
    case String s -> System.out.println(s.toUpperCase());
    case Integer i -> System.out.println(i + 10);
    default -> System.out.println("Unsupported type");
}
```

Exercise 02 – PatternVsInstanceof.java
Write two methods — one using traditional `instanceof` and another using **pattern matching in switch** — to print information about different object types. Compare readability.

Null Handling in Switch Patterns

Key Points

• `switch` now supports explicit `case null`.
• If no `case null` is present, a `NullPointerException` occurs.
• Makes switch safer and more expressive.

Code Snippet

```
Object val = null;
switch (val) {
    case null -> System.out.println("Null detected");
    default -> System.out.println("Non-null value");
}
```

Exercise 03 – SwitchNullCase.java

Write a switch that safely handles null input values and prints “Null input not allowed” when `null` is passed.

Type Patterns in Switch

Key Points

• A pattern can specify both type and binding variable.
• Variable is automatically cast to that type.
• Useful for polymorphic structures.

Code Snippet

```
Object shape = new Rectangle(5, 10);
switch (shape) {
    case Circle c -> System.out.println("Circle radius: " + c.radius());
    case Rectangle r -> System.out.println("Rectangle area: " + r.length()
* r.width());
    default -> System.out.println("Unknown shape");
}
```

Exercise 04 – ShapePatternMatch.java

Create `Shape`, `Circle`, and `Rectangle` classes. Use **pattern matching in switch** to print the area or radius depending on the type of shape.

Guarded Patterns (when clause)

Key Points

• Use `when` to add an extra condition on a matched pattern.
• Provides more precise matching.

Code Snippet

```
Object obj = 42;
switch (obj) {
    case Integer i when i > 100 -> System.out.println("Large number");
    case Integer i -> System.out.println("Small number: " + i);
    default -> System.out.println("Not an integer");
}
```

Exercise 05 – GuardedPattern.java

Write a program that uses a **guarded pattern** to classify integers as “Negative”, “Small”, or “Large”.

Exhaustiveness and Dominance Rules

Key Points

• All possible types must be covered — compiler enforces exhaustive checks.
• More specific patterns must appear before general ones.
• Unreachable patterns cause compilation errors (dominance rule).

Code Snippet

```
switch (obj) {
    case String s when s.isEmpty() -> System.out.println("Empty string");
    case String s -> System.out.println("Non-empty string: " + s);
    default -> System.out.println("Other type");
}
```

Exercise 06 – DominanceRules.java

Try swapping the order of `case String s` and `case String s when s.isEmpty()` and observe the **compiler error**. Fix it by applying the dominance rule.

Sealed Classes and Pattern Matching

Key Points

• Integrates seamlessly with sealed class hierarchies.
• Compiler verifies all subclasses are covered.
• Removes need for `default` in exhaustive switches.

Code Snippet

```
sealed interface Shape permits Circle, Rectangle {}
record Circle(double radius) implements Shape {}
record Rectangle(double length, double width) implements Shape {}

Shape s = new Circle(3.5);
String result = switch (s) {
    case Circle c -> "Circle with radius " + c.radius();
    case Rectangle r -> "Rectangle with area " + (r.length() * r.width());
};
System.out.println(result);
```

Exercise 07 – SealedShapeSwitch.java

Define a sealed hierarchy of `Shape` types and use **pattern matching switch** to describe each subtype without a default branch.

Combining Patterns

Key Points
• Multiple patterns can be combined using commas (multi-label).
• Useful when multiple types share the same behavior.

Code Snippet

```
switch (obj) {
    case Integer i, Long l -> System.out.println("Numeric type");
    case String s -> System.out.println("Text type");
    default -> System.out.println("Other type");
}
```

Exercise 08 – MultiLabelPattern.java
Write a program that identifies whether an object is **text**, **numeric**, or **other**, using **multi-label pattern matching**.

Flow Scoping and Pattern Reuse

Key Points
• Variables introduced in pattern cases are scoped inside the case.
• Cannot be accessed outside the matched case.
• Allows safe reuse of variable names across patterns.

Code Snippet

```
switch (obj) {
    case String s -> System.out.println("String length: " + s.length());
    case Integer s -> System.out.println("Integer value: " + s);
    default -> System.out.println("Unknown");
}
```

Exercise 09 – FlowScope.java
Experiment by declaring the same variable name in multiple case patterns and verify that they work safely due to **flow scoping**.

Key Takeaways

• Pattern matching in switch simplifies type checks and casting.
• Supports `case null`, guarded patterns (`when`), and sealed type hierarchies.
• Improves readability and safety with exhaustiveness checking.
• Available as a preview feature in Java 21 — must enable with `--enable-preview` during compilation and runtime.

Module – Loop Constructs: for, Enhanced for, while, do–while, break, continue

What Are Loops?

Key Points

• Loops execute a block of code repeatedly until a condition is met.
• Reduce code duplication by automating repetitive tasks.
• Types of loops in Java:

1. for loop
2. Enhanced for (for-each)
3. while loop
4. do–while loop

Code Snippet

```
System.out.println("Counting 1 to 5:");
for (int i = 1; i <= 5; i++) {
    System.out.println(i);
}
```

Exercise 01 – LoopBasics.java

Write a program that prints numbers from **1 to 10** using a `for` loop. Modify it to print them in **reverse order** using another loop.

The for Loop

Key Points

• Used when number of iterations is known.
• Structure: `for (initialization; condition; update)`
• Each iteration → check condition → execute body → update variable.

Code Snippet

```
for (int i = 1; i <= 3; i++) {
    System.out.println("Iteration: " + i);
}
```

Flow: Initialization → Condition check → Body → Update → Repeat until false.

Exercise 02 – ForLoopCounter.java

Write a `for` loop that prints all **even numbers between 1 and 20**. Add a counter to display how many even numbers were found.

Multiple Variables and Infinite Loops

Key Points

• `for` can handle multiple variables separated by commas.
• Condition can be omitted → infinite loop (use with `break`).

Code Snippet

```
for (int i = 1, j = 5; i <= j; i++, j--) {
    System.out.println(i + " " + j);
}

for (;;) { // infinite loop
    System.out.println("Running...");
    break;
}
```

Exercise 03 – ForLoopVariants.java

1. Create a `for` loop with **two variables (x, y)** moving in opposite directions.
2. Then create an **infinite loop** that runs until user input is “exit”.

Enhanced for (For-Each Loop)

Key Points

• Introduced in Java 5 for arrays and collections.
• Automatically iterates through elements without index.
• Read-only access to elements.

Code Snippet

```
String[] names = {"Alice", "Bob", "Charlie"};
for (String name : names) {
    System.out.println("Hello, " + name);
}
```

Limitations:
• Cannot modify collection size.
• No index variable access.

Exercise 04 – ForEachNames.java

Create an array of student names. Use an **enhanced for loop** to greet each student. Then, using a normal `for` loop, print each student's index alongside their name.

The while Loop

Key Points

• Used when number of iterations is unknown.
• Checks condition before executing the loop body.
• May not execute even once if condition is false initially.

Code Snippet

```
int i = 1;
while (i <= 3) {
    System.out.println("Count: " + i);
    i++;
}
```

Exercise 05 – WhileLoopSum.java

Write a program using a **while loop** to calculate the **sum of numbers from 1 to N**, where N is entered by the user.

The do–while Loop

Key Points

• Executes at least once, even if condition is false.
• Checks condition after executing loop body.
• Useful for user-input or menu-driven programs.

Code Snippet

```
int count = 1;
do {
    System.out.println("Iteration: " + count);
    count++;
} while (count <= 3);
```

Exercise 06 – DoWhileMenu.java

Create a **menu-driven calculator** using a `do-while` loop that continues until the user selects **"Exit"**.

break Statement

Key Points

• Terminates the innermost loop immediately.
• Control moves to the first statement after the loop.

Code Snippet

```
for (int i = 1; i <= 5; i++) {
    if (i == 3) break;
    System.out.println(i);
}
System.out.println("Loop exited");
```

Exercise 07 – BreakExample.java

Write a loop that prints numbers from 1 to 100 but **stops if the number reaches 42**.
Print a message indicating where the loop stopped.

continue Statement

Key Points
• Skips current iteration and moves to next iteration.
• Condition check or update expression still occurs.

Code Snippet

```
for (int i = 1; i <= 5; i++) {
    if (i == 3) continue;
    System.out.println(i);
}
```

Output:
1
2
4
5

Exercise 08 – ContinueExample.java
Write a loop that prints numbers from 1 to 20 but **skips all multiples of 3** using `continue`.

Nested Loops

Key Points
• A loop inside another loop.
• Inner loop executes fully for every iteration of outer loop.
• Commonly used for matrices or pattern generation.

Code Snippet

```
for (int i = 1; i <= 3; i++) {
    for (int j = 1; j <= 3; j++) {
        System.out.print(i + "," + j + " ");
    }
    System.out.println();
}
```

Exercise 09 – NestedPattern.java
Use **nested loops** to print a right-angled triangle of stars:

```
*
**
***
****
*****
```

Loop Control Flow Summary

Loop Type	Condition Checked	Guaranteed Execution	Use Case
for	Before	No	Fixed iterations
Enhanced for	Implicit	No	Collections/arrays
while	Before	No	Conditional repetition
do–while	After	Yes	At least one run
break	N/A	—	Exit loop early
continue	N/A	—	Skip to next iteration

Exercise 10 – LoopSummaryDemo.java
Write one small example of each loop type (`for`, `while`, `do-while`, `enhanced for`) to demonstrate their execution difference.

Key Takeaways

• `for` → best for fixed counts.
• `while` / `do-while` → best for condition-based repetition.
• `enhanced for` → ideal for collections and arrays.
• `break` → exits loop; `continue` → skips iteration.
• Combine loops with conditionals for dynamic flow control.

Module – Labelled Loops and Nested Control Structures

What Are Labelled Loops?

Key Points

• A label is an identifier followed by a colon (:) placed before a loop.
• Used to control flow in nested loops — allows `break` or `continue` to target a specific loop.
• Improves clarity in deeply nested logic.

Code Snippet

```
outer:
for (int i = 1; i <= 3; i++) {
    for (int j = 1; j <= 3; j++) {
        if (j == 2) break outer;
        System.out.println(i + " " + j);
    }
}
System.out.println("Exited outer loop");
```

Exercise 01 – LabelledLoopIntro.java

Write a program with two nested loops labeled `outer` and `inner`.
Break out of both loops when `i * j` equals 6, and print where it stopped.

Why Use Labels?

Key Points

• In nested loops, `break` or `continue` affects only the innermost loop by default.
• Labels allow breaking or continuing an outer loop directly.
• Improves control in multi-dimensional or search-based logic (e.g., matrix scans).

Diagram

```
outer: for (...)
          inner: for (...)
                    if(condition) break outer;
```

Exercise 02 – LabelPurpose.java

Demonstrate the difference between a **normal break** and a **labelled break** inside nested loops.
Print iteration logs to observe the difference.

Labelled break

Key Points
• Terminates the labelled loop immediately.
• All inner loops inside it are exited as well.

Code Snippet

```
search:
for (int i = 0; i < 3; i++) {
    for (int j = 0; j < 3; j++) {
        if (i == j && i == 1) {
            System.out.println("Found match at " + i + "," + j);
            break search; // exits both loops
        }
    }
}
System.out.println("Search completed");
```

Exercise 03 – LabelledBreak.java
Create a nested loop with a label `search:` that scans a 2D array for a specific value.
Use `break search;` once the value is found and display its location.

Labelled continue

Key Points
• Skips to the next iteration of the labelled loop.
• Useful when a condition invalidates the current outer iteration.

Code Snippet

```
outer:
for (int i = 1; i <= 3; i++) {
    for (int j = 1; j <= 3; j++) {
        if (j == 2) continue outer; // skip current i iteration
        System.out.println("i=" + i + ", j=" + j);
    }
}
```

Output:

```
i=1, j=1
i=2, j=1
i=3, j=1
```

Exercise 04 – LabelledContinue.java
Write a program where the **outer loop** runs students, and the **inner loop** runs subjects.
Use `continue outer;` to skip a student entirely if any subject is marked “Fail”.

Label Scope and Rules

Key Points

• A label name must be a valid identifier (no keywords, no spaces).
• Must be placed immediately before a loop statement.
• Labels cannot be applied to arbitrary blocks `{ }`.
• `break` or `continue` can reference only enclosing labels.

Invalid Example

```
myLabel: {
    System.out.println("Not a loop!");
    break myLabel; // ❌ Compile-time error
}
```

Exercise 05 – LabelScope.java

Try applying a label to a code block instead of a loop and observe the **compiler error**.
Then fix it by correctly labeling a `for` or `while` loop.

Nested Control Structures

Key Points

• Combining loops and conditionals increases control flow flexibility.
• Nested structures can mix `if`, `while`, `for`, and `switch`.
• Used in algorithms like searching, sorting, matrix operations, and menu-driven programs.

Code Snippet

```
for (int i = 1; i <= 3; i++) {
    if (i % 2 == 0) {
        for (int j = 1; j <= 2; j++) {
            System.out.println("Even outer: " + i + ", inner: " + j);
        }
    } else {
        System.out.println("Odd outer: " + i);
    }
}
```

Exercise 06 – NestedConditionals.java

Write a nested loop that prints only **even outer numbers** and their **inner loop multiplications**.
Add an `if` condition inside to skip printing results where product > 4.

Combining Labels with Nested Control

Key Points
• Labelled control statements simplify complex nested logic.
• Useful in early exits when certain conditions are met deep inside nested loops.

Code Snippet

```
outer:
for (int row = 1; row <= 3; row++) {
    for (int col = 1; col <= 3; col++) {
        if (row * col == 4) {
            System.out.println("Condition met at " + row + "," + col);
            break outer;
        }
    }
}
System.out.println("Exited at condition match");
```

Exercise 07 – BreakOnCondition.java
Modify the code to **break outer** when the product equals **9** and print the exact `(row, col)` pair where it occurred.

Real-World Use Case: Searching in a 2D Array

Key Points
• Labelled loops help when scanning multi-dimensional data.
• Breaks out early upon finding a match.

Code Snippet

```
int[][] matrix = {
    {10, 20, 30},
    {40, 50, 60},
    {70, 80, 90}
};

find:
for (int i = 0; i < matrix.length; i++) {
    for (int j = 0; j < matrix[i].length; j++) {
        if (matrix[i][j] == 50) {
            System.out.println("Found at (" + i + "," + j + ")");
            break find;
        }
    }
}
```

Exercise 08 – MatrixSearch.java
Write a labeled loop that searches a **2D matrix** for a given number.
If the number is found, print its indices and break from both loops immediately.

Best Practices for Nested Control Structures

Key Points

• Avoid excessive nesting — it reduces readability.
• Prefer methods or early returns instead of deep loops where possible.
• Labelled loops should be used sparingly and only when needed.
• Keep naming meaningful: `search:`, `outer:`, `scan:`.

Exercise 09 – NestedBestPractices.java

Refactor a deeply nested loop structure into **two methods** — one to search and one to print results.
Use a label only where logically necessary.

Key Takeaways

• Labels extend control flow across multiple nested loops.
• `break <label>` → exits labelled loop entirely.
• `continue <label>` → skips to next iteration of labelled loop.
• Nested control structures allow multi-level decision and iteration logic.
• Use labels wisely for clarity and maintainability.

Module – The Role of return in Flow Control

Introduction to return

Key Points

• `return` is a flow-control statement that exits a method immediately.
• It optionally sends a value back to the caller.
• Every method has at least one possible return point — either explicit or implicit (for void methods).

Code Snippet

```
void greet() {
    System.out.println("Hello!");
    return; // exits method
}
```

Exercise 01 – ReturnIntro.java

Create a method `displayMessage()` that prints “Welcome!” and exits using `return;`.
Then call it twice and observe that the second call executes independently.

Return in Non-Void Methods

Key Points

• Non-void methods must explicitly return a value matching the declared return type.
• All possible execution paths must have a return statement.

Code Snippet

```
int add(int a, int b) {
    return a + b; // must return int
}

// Invalid Example
int calculate(int n) {
    if (n > 0)
        return n;
    // ❌ Missing return for negative n
}
```

Exercise 02 – ReturnInMethods.java

Write a method `max(int a, int b)` that returns the greater of the two numbers.
Ensure all code paths end with a return statement.

Return in void Methods

Key Points

• For `void` methods, `return;` is optional — used only for early exits.
• `return` cannot send a value in a void method.

Code Snippet

```
void process(int n) {
    if (n < 0) {
        System.out.println("Invalid input");
        return; // exit early
    }
    System.out.println("Processing: " + n);
}
```

Exercise 03 – VoidReturn.java

Write a method `printSquare(int n)` that:
• Prints an error and exits early if `n` is negative.
• Otherwise, prints the square of `n`.

Return as Early Exit (Guard Clause)

Key Points

• `return` is often used to short-circuit logic.
• Improves readability by handling invalid or special cases early.
• Avoids deep nesting of `if-else` blocks.

Code Snippet

```
boolean isValidAge(int age) {
    if (age < 0) return false;
    if (age > 120) return false;
    return true;
}
```

Flow:
• Returns immediately once a condition fails.
• Prevents unnecessary execution of remaining code.

Exercise 04 – EarlyExit.java

Write a method `isEligible(int age)` that:
• Returns `false` if age < 18 or age > 60.
• Returns `true` otherwise.
Use early return instead of nested if–else.

Return in Conditional Structures

Key Points

• Multiple `return` statements can exist in one method.
• The first encountered return transfers control back to caller.

Code Snippet

```
String checkNumber(int n) {
    if (n > 0) return "Positive";
    else if (n < 0) return "Negative";
    else return "Zero";
}
```

Exercise 05 – MultiReturn.java

Write a method `grade(int score)` that returns:
• “Excellent” if ≥ 90
• “Good” if ≥ 75
• “Average” if ≥ 50
• “Fail” otherwise

Return in Loops

Key Points

• `return` immediately exits both the loop and the method.
• Use carefully to avoid unexpected flow termination.

Code Snippet

```
boolean contains(int[] arr, int key) {
    for (int num : arr) {
        if (num == key)
            return true; // exits loop + method
    }
    return false;
}
```

Exercise 06 – ReturnInLoop.java

Write a method `findFirstEven(int[] arr)` that returns the **first even number** in an array using `return`.
If no even number is found, return `-1`.

Return in Recursive Methods

Key Points

• In recursion, `return` provides both termination and value propagation.
• Each call returns its result to the previous one in the call stack.

Code Snippet

```
int factorial(int n) {
    if (n == 1) return 1;          // base case
    return n * factorial(n - 1);   // recursive return
}
```

Flow:
Each recursive call pauses until its inner call completes, then multiplies and returns the result.

Exercise 07 – RecursiveReturn.java

Implement a recursive method `sumToN(int n)` that returns the sum of numbers from 1 to n using recursive returns.

Interaction with try–catch–finally

Key Points

• `return` inside try or catch executes before the `finally` block runs.
• The `finally` block executes even if `return` is used.

Code Snippet

```
int compute() {
    try {
        return 10;
    } finally {
        System.out.println("Finally block executed");
    }
}
```

Output:

```
Finally block executed
```

Exercise 08 – ReturnWithFinally.java

Write a method `getValue()` that returns an integer inside a `try` block.
Add a `finally` block to print “Cleanup done” and verify it executes even when `return` is called.

Restrictions and Best Practices

Key Points

• `return` cannot appear outside a method or constructor.
• Constructors never return a value — not even `void`.
• Too many return statements can reduce readability — structure them logically.
• Use return early for validation, late for results.

Invalid Example

```
class Test {
    Test() {
        return; // ❌ not allowed in constructors
    }
}
```

Exercise 09 – ReturnRules.java

Try placing `return` inside a constructor and observe the compilation error.
Then refactor your logic to use a **guard method** instead of returning from constructor.

Key Takeaways

- `return` transfers control back to caller, optionally with a value.
- Can be used in void methods for early exit.
- Terminates both loop and method when triggered inside loops.
- In recursion, defines both base condition and result propagation.
- `finally` always executes even if `return` is encountered in try or catch.
- Write clear, structured returns for readability and maintainability.

Module – Exception Hierarchy (Throwable, Exception, RuntimeException, Error)

What Are Exceptions?

Key Points

• An exception is an event that disrupts normal program flow.
• Occurs during runtime due to unexpected conditions (e.g., divide by zero, invalid input).
• Java handles these gracefully through a structured exception hierarchy under `Throwable`.

Code Snippet

```
int x = 10, y = 0;
int result = x / y; // throws ArithmeticException
System.out.println("This line never executes");
```

Exercise 01 – BasicExceptionDemo.java

Write a program that divides two integers.
Trigger an `ArithmeticException` by dividing by zero and observe the runtime error message.

Top of the Hierarchy: Throwable

Key Points

• `Throwable` is the root class for all errors and exceptions.
• Two main branches:

1. `Error` → serious system failures (not meant to be caught).
2. `Exception` → recoverable program-level problems.

Hierarchy Diagram

```
java.lang.Object
        |
   Throwable
   /       \
Error     Exception
```

Exercise 02 – ThrowableHierarchy.java

Create an example that throws both an `Error` and an `Exception`.
Print their class names using `getClass().getName()` to confirm both extend `Throwable`.

The Error Class

Key Points
• Represents serious system-level problems beyond application control.
• Typically caused by the JVM or environment.
• Should not be caught or handled in most cases.

Common Subclasses
• `OutOfMemoryError`
• `StackOverflowError`
• `VirtualMachineError`
• `AssertionError`

Code Snippet

```
try {
    recursive(); // may cause StackOverflowError
} catch (Error e) {
    System.out.println("Caught error: " + e);
}

void recursive() { recursive(); }
```

Exercise 03 – ErrorDemo.java
Deliberately cause a `StackOverflowError` using recursion and catch it.
Print a message to indicate how severe system errors can terminate execution.

The Exception Class

Key Points

• Represents conditions that a program might want to handle.
• Divided into two categories:
• Checked exceptions → must be declared or handled using `throws` or `try-catch`.
• Unchecked exceptions → subclasses of `RuntimeException`, don't require declaration.

Hierarchy Diagram

```
Throwable
 ├── Error
 └── Exception
      ├── IOException
      ├── SQLException
      ├── ClassNotFoundException
      └── RuntimeException
```

Exercise 04 – ExceptionCategories.java

List three checked and three unchecked exceptions in Java.
Classify each correctly under Exception or RuntimeException.

Checked Exceptions

Key Points
• Must be caught or declared in the method signature.
• Checked by the compiler at compile-time.
• Used for expected, recoverable problems (like file or network issues).

Examples
• `IOException`
• `SQLException`
• `ParseException`

Code Snippet

```
import java.io.*;
void readFile() throws IOException {
    BufferedReader br = new BufferedReader(new FileReader("data.txt"));
    System.out.println(br.readLine());
    br.close();
}
```

Exercise 05 – CheckedExceptionDemo.java
Write a method that reads a non-existent file using `FileReader`.
Observe the compile-time error if `throws IOException` is not added.

Unchecked Exceptions (RuntimeException)

Key Points

• Subclasses of `RuntimeException`.
• Not checked at compile-time — may occur due to programming errors.
• Usually caused by logic mistakes, invalid input, or illegal state.

Common Examples

• `NullPointerException`
• `ArithmeticException`
• `ArrayIndexOutOfBoundsException`
• `IllegalArgumentException`

Code Snippet

```
String s = null;
System.out.println(s.length()); // NullPointerException
```

Exercise 06 – UncheckedExceptionDemo.java

Write separate code blocks that intentionally trigger a `NullPointerException`, `ArrayIndexOutOfBoundsException`, and `ArithmeticException`.
Run them to see how unchecked exceptions behave at runtime.

Comparison: Checked vs Unchecked

Feature	Checked Exception	Unchecked Exception
Compile-time check	✅ Required	❌ Not required
Subclass of	`Exception` (not `RuntimeException`)	`RuntimeException`
Handling	Must be caught or declared	Optional
Cause	External factors (I/O, DB)	Programming errors
Example	`IOException`, `SQLException`	`NullPointerException`, `ArithmeticException`

Exercise 07 – ExceptionComparison.java
Write two methods:

1. One that throws a checked exception (`IOException`).
2. One that throws an unchecked exception (`NullPointerException`).
   Call both and analyze compiler behavior.

Custom Exceptions

Key Points
• Developers can create their own exception classes.
• Extend from `Exception` for checked exceptions, or `RuntimeException` for unchecked.

Code Snippet

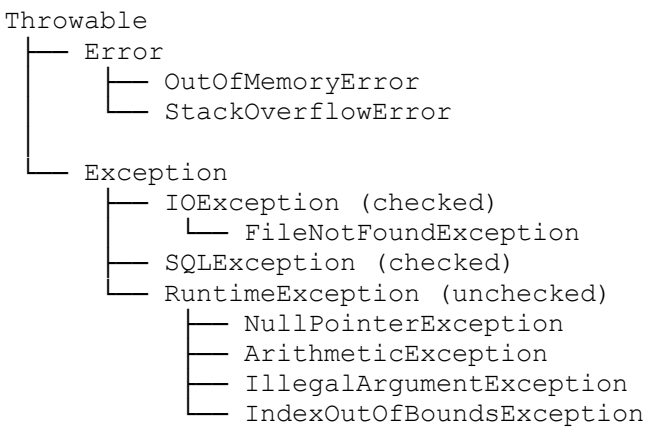
```
class InvalidAgeException extends Exception {
    InvalidAgeException(String message) {
        super(message);
    }
}

void register(int age) throws InvalidAgeException {
    if (age < 18)
        throw new InvalidAgeException("Age must be 18 or above");
}
```

Exercise 08 – CustomException.java
Create a custom exception `LowBalanceException` for a banking app.
Throw it when balance < 1000 and handle it gracefully with a message.

Nested Hierarchy Summary

Diagram



Exercise 09 – ExceptionTreePrint.java
Write a small program that prints a simplified version of the Exception hierarchy using indentation.
Display at least 3 levels of the tree.

Key Takeaways

• All exceptions and errors derive from `Throwable`.
• `Error` → unrecoverable; `Exception` → recoverable.
• Checked exceptions → must be handled or declared.
• Unchecked exceptions (`RuntimeException`) → optional handling.
• Use custom exceptions for domain-specific validation or business rules.
• Handle only those exceptions you can recover from — let the rest propagate.

Module – Checked vs Unchecked Exceptions

Introduction

Key Points

• All exceptions derive from `Throwable`.
• They are divided into two main categories based on compile-time checking:

1. Checked exceptions – must be declared or handled.
2. Unchecked exceptions – do not require explicit handling.
   • This distinction affects how and when exceptions are detected and managed.

Exercise 01 – ExceptionIntro.java
Write a program that demonstrates one **checked** (`IOException`) and one **unchecked** (`NullPointerException`) exception.
Observe how the compiler reacts differently to each.

What Are Checked Exceptions?

Key Points

• Checked at compile time by the Java compiler.
• Represent recoverable situations that the program should anticipate and handle.
• Must be:
• Enclosed in a try–catch, or
• Declared using `throws` in the method signature.

Common Examples:

- `IOException`
- `SQLException`
- `FileNotFoundException`
- `ParseException`

Code Snippet

```
import java.io.*;

void readFile() throws IOException {
    BufferedReader br = new BufferedReader(new FileReader("data.txt"));
    System.out.println(br.readLine());
    br.close();
}
```

Exercise 02 – CheckedExceptionDemo.java

Create a method that opens a file named `"input.txt"`.
Handle the exception using both:

1. `try-catch` block, and
2. `throws` declaration.
   Compare compile-time behavior in both approaches.

What Are Unchecked Exceptions?

Key Points

• Not checked at compile time — occur during runtime.
• Represent programming or logical errors.
• Subclasses of `RuntimeException`.
• Do not require `throws` or `try-catch` handling.

Common Examples:

• `NullPointerException`
• `ArithmeticException`
• `ArrayIndexOutOfBoundsException`
• `IllegalArgumentException`

Code Snippet

```
int[] arr = {1, 2, 3};
System.out.println(arr[5]); // ArrayIndexOutOfBoundsException
```

Exercise 03 – UncheckedExceptionDemo.java

Write a method that causes three runtime errors —

1. `NullPointerException`,
2. `ArithmeticException`,
3. `ArrayIndexOutOfBoundsException`.
   Run them and observe which ones the compiler ignores.

Compiler Enforcement

Key Points

• Compiler enforces handling for **checked exceptions only**.
• Unchecked exceptions are ignored by the compiler, but can still cause program failure.

Code Snippet

```
// Checked exception: must handle or declare
Thread.sleep(1000); // throws InterruptedException

// Unchecked exception: no compile-time error
int result = 10 / 0; // ArithmeticException
```

Exercise 04 – CompileCheck.java

Write one statement that triggers a checked exception (`Thread.sleep`) and one that triggers an unchecked one (`10/0`).
Comment out the try–catch and check which line fails to compile.

When to Use Checked Exceptions

Key Points

• Use for conditions that the caller can recover from.
• Typically external issues:
• File not found
• Network unavailable
• Invalid data input from external source
• Promotes robust and predictable error handling.

Code Snippet

```
try {
    readFile();
} catch (IOException e) {
    System.out.println("File read failed: " + e.getMessage());
}
```

Exercise 05 – FileReadChecked.java

Simulate reading a missing file using `FileReader`.
Handle it gracefully by printing a recovery message instead of crashing.

When to Use Unchecked Exceptions

Key Points

• Use for programming mistakes that should be fixed in code.
• Examples:
• Null references
• Invalid arguments
• Logic errors
• Not meant to be caught — instead, avoided through defensive coding.

Code Snippet

```
void divide(int a, int b) {
    if (b == 0) throw new IllegalArgumentException("Denominator cannot be
zero");
    System.out.println(a / b);
}
```

Exercise 06 – DefensiveCoding.java

Write a method that validates arguments before performing division or string operations. Throw `IllegalArgumentException` if parameters are invalid.

Comparison Table

Feature	Checked Exception	Unchecked Exception
Checked by compiler	✅ Yes	❌ No
Subclass of	`Exception` (not `RuntimeException`)	`RuntimeException`
Declaration required	Yes (`throws` or `try-catch`)	No
When to use	Expected recoverable conditions	Programming or logic errors
Examples	`IOException`, `SQLException`	`NullPointerException`, `ArithmeticException`

Exercise 07 – ExceptionComparison.java
Implement one checked and one unchecked exception in a single program.
Document their compile-time vs runtime handling differences as comments.

Custom Exceptions Example

Key Points

- You can define both checked and unchecked custom exceptions.
- Extending `Exception` → Checked
- Extending `RuntimeException` → Unchecked

Code Snippet

```
// Checked exception
class InvalidAgeException extends Exception {
    InvalidAgeException(String msg) { super(msg); }
}

// Unchecked exception
class InvalidInputException extends RuntimeException {
    InvalidInputException(String msg) { super(msg); }
}
```

Exercise 08 – CustomExceptionDemo.java
Create:

1. A checked exception `LowBalanceException` (extends `Exception`).
2. An unchecked exception `InvalidPinException` (extends `RuntimeException`).
   Throw each under relevant conditions in a banking example.

Best Practices

Key Points

• Use checked exceptions for expected recoverable errors.
• Use unchecked exceptions for programming logic errors.
• Do not overuse checked exceptions — they can clutter code.
• Document all exceptions clearly in method contracts (Javadoc).
• Convert low-level checked exceptions into meaningful, high-level unchecked ones when appropriate.

Exercise 09 – ExceptionStrategy.java

Refactor an I/O method to catch `IOException` and rethrow it as a custom `DataLoadException` (extends `RuntimeException`) with a clear message.

Key Takeaways

• Checked exceptions → enforce explicit handling for recoverable conditions.
• Unchecked exceptions → indicate defects or invalid logic.
• Both types improve robustness when used correctly.
• Checked → handle gracefully; Unchecked → prevent by design.
• A clean exception strategy improves maintainability and clarity of code.

Module – Multi-Catch and Rethrowing Exceptions

Introduction

Key Points

• Real-world programs often face multiple exception types from a single block of code.
• Java provides **multi-catch** to handle different exceptions in one clause.
• You can also **rethrow** an exception to delegate handling to a higher method.

Exercise 01 – MultiCatchIntro.java

Write a simple program that reads a file and parses a number.
Trigger different exceptions (`FileNotFoundException`, `NumberFormatException`) from the same block and observe how many catch blocks are needed without multi-catch.

Traditional Approach (Before Java 7)

Key Points

• Each exception type required a separate catch block.
• This led to repetitive code if handling logic was similar.

Code Snippet

```
try {
    BufferedReader br = new BufferedReader(new FileReader("data.txt"));
    int num = Integer.parseInt(br.readLine());
} catch (FileNotFoundException e) {
    System.out.println("File not found");
} catch (NumberFormatException e) {
    System.out.println("Invalid number format");
}
```

Exercise 02 – TraditionalCatch.java

Implement the above code and note how duplicate handling logic increases verbosity. Refactor later using multi-catch for comparison.

Multi-Catch Syntax (Java 7+)

Key Points

• Multiple exception types can be caught using the **pipe (|)** operator.
• Applicable when handling logic is identical for all listed exceptions.
• Improves code readability and reduces duplication.

Code Snippet

```
try {
    BufferedReader br = new BufferedReader(new FileReader("data.txt"));
    int num = Integer.parseInt(br.readLine());
} catch (FileNotFoundException | NumberFormatException e) {
    System.out.println("Error occurred: " + e.getMessage());
}
```

Exercise 03 – MultiCatchExample.java

Modify your previous program to use **multi-catch**.
Verify that both exceptions are handled by the same block and display the shared message.

Multi-Catch Rules

Key Points

• All caught exception types must be **disjoint** — no subclass–superclass relationship.
• The `|` operator works as an **OR** between exception types.
• Exception variable (`e`) is effectively **final** and cannot be reassigned.

Invalid Example:

```
// ❌ Compile-time error: IOException is superclass of
FileNotFoundException
catch (IOException | FileNotFoundException e) {
    ...
}
```

Valid Example:

```
catch (IOException | SQLException e) {
    ...
}
```

Exercise 04 – MultiCatchRules.java

Try combining related exceptions like `IOException` and `FileNotFoundException` in a multi-catch block.
Observe the compiler error and fix it using valid disjoint exception types.

Practical Multi-Catch Example

Key Points
• Ideal for handling exceptions with shared recovery logic.

Code Snippet

```
try {
    Connection conn = DriverManager.getConnection(url);
    Statement stmt = conn.createStatement();
    stmt.executeUpdate("INSERT INTO students VALUES(1, 'John')");
} catch (SQLException | NullPointerException e) {
    System.out.println("Database operation failed: " + e);
}
```

Output:

```
Database operation failed: java.sql.SQLException: Table not found
```

Exercise 05 – DatabaseMultiCatch.java
Simulate a database operation that can throw `SQLException` or `NullPointerException`. Handle both using multi-catch and log a single, clear message.

Rethrowing Exceptions (Basic)

Key Points
• You can rethrow an exception after partial handling or logging.
• Useful for delegating error handling to the caller.

Code Snippet

```
void readFile() throws IOException {
    try {
        BufferedReader br = new BufferedReader(new FileReader("data.txt"));
        System.out.println(br.readLine());
    } catch (IOException e) {
        System.out.println("Logging error: " + e.getMessage());
        throw e; // rethrow
    }
}
```

Exercise 06 – RethrowBasic.java
Write a method that reads a file and logs the exception before rethrowing it.
Handle the rethrown exception in the caller method.

Rethrowing a New Exception

Key Points

• A new exception can be thrown to wrap another (chained exception).
• Use `throw new ExceptionType("msg", cause)` to preserve the root cause.
• Improves abstraction — hides low-level details from higher layers.

Code Snippet

```
void loadConfig() throws ConfigurationException {
    try {
        Files.readString(Path.of("config.txt"));
    } catch (IOException e) {
        throw new ConfigurationException("Failed to load config", e);
    }
}
```

Output Stack Trace Example:

```
ConfigurationException: Failed to load config
Caused by: java.io.IOException: File not found
```

Exercise 07 – ChainedException.java

Create a custom exception `DataLoadException`.
Catch an `IOException` and rethrow it as `new DataLoadException("Load failed", e)` while preserving the original cause.

More Specific Rethrow Typing (Java 7+)

Key Points
• The compiler can infer the specific exception type being rethrown.
• You can rethrow without losing the original type information.

Code Snippet

```
void process() throws IOException, SQLException {
    try {
        if (new Random().nextBoolean())
            throw new IOException("File issue");
        else
            throw new SQLException("DB issue");
    } catch (Exception e) {
        System.out.println("Partial handling...");
        throw e; // compiler infers IOException | SQLException
    }
}
```

Exercise 08 – SpecificRethrow.java
Create a method that can throw both `IOException` and `SQLException`.
Handle them in a single catch and rethrow.
Inspect the method signature to verify compiler inference.

Combining Multi-Catch and Rethrow

Key Points
• You can combine both concepts for cleaner and layered exception handling.

Code Snippet

```
void performOperation() throws IOException, SQLException {
    try {
        executeTask();
    } catch (IOException | SQLException e) {
        System.out.println("Operation failed: " + e.getMessage());
        throw e; // rethrow to propagate upward
    }
}
```

Use Case:
• Local logging or rollback at one level.
• Final handling or user message at another level.

Exercise 09 – MultiCatchRethrow.java
Write a method that performs multiple operations (`readFile`, `updateDB`) and may throw `IOException` or `SQLException`.
Use **multi-catch + rethrow** to propagate exceptions upward after logging.

Best Practices

Key Points

• Use multi-catch when handling logic is identical.
• Never catch broad exceptions like `Exception` or `Throwable` unless necessary.
• Use rethrow to delegate responsibility, not to hide problems.
• Wrap low-level exceptions with meaningful business exceptions when needed.
• Log before rethrowing to preserve traceability.

Exercise 10 – ExceptionBestPractices.java

Refactor an existing exception-handling method to:

1. Avoid catching `Exception` directly.
2. Use multi-catch properly.
3. Log before rethrowing a custom business exception.

Key Takeaways

• **Multi-catch (`|`)** simplifies code when handling similar exceptions.
• **Rethrowing** lets you escalate exceptions to higher layers responsibly.
• Compiler enforces safe typing for multi-catch and rethrow since Java 7.
• Use **exception chaining (`new Exception(cause)`)** to preserve root context.
• Always balance simplicity (multi-catch) with clarity (separate handlers when logic differs).

Module – try-with-resources and the AutoCloseable Interface

Introduction

Key Points

• Resource management (like files, DB connections) requires proper closing to avoid memory leaks.
• Traditionally handled using try–catch–finally blocks.
• From Java 7, the try-with-resources statement automates this cleanup.
• Any object implementing AutoCloseable or Closeable can be managed automatically.

Exercise 01 – ResourceIntro.java

Write a small program that reads a file using BufferedReader. Close it manually using finally. Then refactor using try-with-resources and compare readability.

Traditional Resource Handling

Key Points
• In older Java versions, resources had to be closed manually in finally.
• Prone to errors if exceptions occurred before closing.

Code Snippet

```
BufferedReader br = null;
try {
    br = new BufferedReader(new FileReader("data.txt"));
    System.out.println(br.readLine());
} catch (IOException e) {
    e.printStackTrace();
} finally {
    try {
        if (br != null)
            br.close();
    } catch (IOException e) {
        e.printStackTrace();
    }
}
```

Exercise 02 – ManualResourceClose.java
Recreate the above example and intentionally throw an exception before closing. Observe what happens when finally is skipped.

try-with-resources Simplified

Key Points

• Introduced in Java 7.
• Automatically closes resources when the try block finishes (success or failure).
• Eliminates the need for a finally block.

Code Snippet

```
try (BufferedReader br = new BufferedReader(new FileReader("data.txt"))) {
    System.out.println(br.readLine());
} catch (IOException e) {
    e.printStackTrace();
}
```

Exercise 03 – TryWithResourcesDemo.java

Refactor the previous program using try-with-resources. Verify that the resource closes automatically without finally.

Multiple Resources in try-with-resources

Key Points
• You can manage multiple resources in one try statement.
• Resources are closed in reverse order of their declaration.

Code Snippet

```
try (
    FileInputStream fis = new FileInputStream("input.txt");
    FileOutputStream fos = new FileOutputStream("output.txt")
) {
    fos.write(fis.readAllBytes());
} catch (IOException e) {
    System.out.println("I/O Error: " + e.getMessage());
}
```

Closure order: fos.close() → fis.close()

Exercise 04 – MultiResourceCopy.java
Create a file copy program using FileInputStream and FileOutputStream in one try block.
Add print statements in custom subclasses to verify reverse close order.

The AutoCloseable Interface

Key Points

• Any class implementing AutoCloseable can be used in try-with-resources.
• Defines a single method: `void close() throws Exception;`
• java.io.Closeable (used in I/O streams) is a subinterface of AutoCloseable.

Example

```
class MyResource implements AutoCloseable {
    public void use() { System.out.println("Using resource"); }
    public void close() { System.out.println("Resource closed"); }
}

try (MyResource r = new MyResource()) {
    r.use();
}
```

Output:
Using resource
Resource closed

Exercise 05 – AutoCloseableDemo.java

Create a custom class implementing AutoCloseable that prints messages when used and closed. Verify automatic cleanup inside try-with-resources.

Custom AutoCloseable Resources

Key Points
• You can define custom cleanup logic inside close().
• Useful for database connections, file handles, or network sockets.

Code Snippet

```
class DatabaseConnection implements AutoCloseable {
    void connect() { System.out.println("Connected to DB"); }
    public void close() { System.out.println("DB connection closed"); }
}

try (DatabaseConnection db = new DatabaseConnection()) {
    db.connect();
}
```

Output:
Connected to DB
DB connection closed

Exercise 06 – CustomResource.java
Define a class TempFileHandler that creates and deletes a temp file automatically. Use it in try-with-resources to verify cleanup.

Suppressed Exceptions

Key Points
• If both the try block and close() throw exceptions, the close exception is suppressed.
• Access suppressed exceptions using Throwable.getSuppressed().

Code Snippet

```
class FaultyResource implements AutoCloseable {
    public void close() throws Exception {
        throw new Exception("Error closing resource");
    }
}

try (FaultyResource fr = new FaultyResource()) {
    throw new Exception("Error in try block");
} catch (Exception e) {
    System.out.println("Caught: " + e.getMessage());
    for (Throwable t : e.getSuppressed()) {
        System.out.println("Suppressed: " + t);
    }
}
```

Output:
Caught: Error in try block
Suppressed: java.lang.Exception: Error closing resource

Exercise 07 – SuppressedExceptionDemo.java
Write a program where both try and close() throw exceptions. Print all suppressed exceptions using getSuppressed().

try-with-resources with Catch and Finally

Key Points
• You can still include catch and finally with try-with-resources.
• Resources are closed before the catch or finally executes.

Code Snippet

```
try (BufferedReader br = new BufferedReader(new FileReader("data.txt"))) {
    System.out.println(br.readLine());
} catch (IOException e) {
    System.out.println("Error reading file");
} finally {
    System.out.println("Operation complete");
}
```

Execution order: try → resource closed → catch/finally

Exercise 08 – TryWithCatchFinally.java
Print messages or timestamps in try, catch, and finally blocks to confirm closure occurs before catch/finally executes.

Improvements in Java 9+

Key Points

• In Java 9+, existing resources can be used without redeclaring them inside try().
• They just need to be effectively final.

Code Snippet

```
BufferedReader br = new BufferedReader(new FileReader("data.txt"));
try (br) {
    System.out.println(br.readLine());
}
```

Exercise 09 – Java9ResourceReuse.java

Create a BufferedReader outside the try block and reuse it in try(br). Verify that it closes automatically.

Comparison: Manual vs try-with-resources

Aspect	Manual Close (finally)	try-with-resources
Boilerplate	High	Low
Safety	Error-prone	Automatic
Exception handling	Requires nesting	Simplified
Suppressed exceptions	Manual	Automatic
Readability	Low	High

Exercise 10 – ComparisonDemo.java
Implement both manual and try-with-resources versions of file reading. Compare line counts and clarity.

Key Takeaways

• try-with-resources automatically closes all resources implementing AutoCloseable.
• Reduces boilerplate and improves reliability.
• Resources are closed in reverse order of declaration.
• Suppressed exceptions preserve secondary close-time errors.
• From Java 9 onward, effectively final resources can be reused inside try().

Module – Creating Custom Exceptions and Chaining Causes

Why Create Custom Exceptions

Key Points
• Built-in exceptions (like IOException, NullPointerException) may not express business-specific errors.
• Custom exceptions make your code self-explanatory and domain-relevant.
• They improve readability, maintainability, and debugging clarity.
Examples of domain-specific exceptions:
• InsufficientFundsException in banking apps
• InvalidUserInputException in validation logic
• DataNotFoundException in data services

Exercise 01 – DomainSpecificException.java
Create a custom exception named `InvalidOrderStateException` for an e-commerce order workflow. Throw it when an invalid state transition occurs.

Designing a Custom Checked Exception

Key Points

• Extend the Exception class for checked exceptions (must handle or declare).
• Use constructors to pass custom messages or causes.

Code Snippet

```
class InvalidAgeException extends Exception {
    InvalidAgeException(String message) {
        super(message);
    }
    InvalidAgeException(String message, Throwable cause) {
        super(message, cause);
    }
}

void register(int age) throws InvalidAgeException {
    if (age < 18)
        throw new InvalidAgeException("Age must be 18 or above");
}
```

Exercise 02 – CheckedCustomException.java

Write a method that validates user registration age. Throw a custom `InvalidAgeException` if the age is below 18 and handle it in main().

Designing a Custom Unchecked Exception

Key Points

• Extend RuntimeException for unchecked exceptions (no need to declare).
• Used when error is due to programming mistake rather than recoverable condition.

Code Snippet

```
class InvalidInputException extends RuntimeException {
    InvalidInputException(String message) {
        super(message);
    }
    InvalidInputException(String message, Throwable cause) {
        super(message, cause);
    }
}

void divide(int a, int b) {
    if (b == 0)
        throw new InvalidInputException("Division by zero not allowed");
    System.out.println(a / b);
}
```

Exercise 03 – UncheckedCustomException.java

Create a `NegativeNumberException` extending RuntimeException. Use it in a method that rejects negative numbers in an array and throws the exception when found.

Choosing Between Checked and Unchecked

Aspect	Checked Exception	Unchecked Exception
Extend from	Exception	RuntimeException
Must declare/handle	✅ Yes	❌ No
Represents	Recoverable conditions	Programming logic errors
Example	InvalidAgeException	InvalidInputException

Exercise 04 – ExceptionChoice.java
List three cases where checked exceptions are suitable and three where unchecked exceptions make more sense. Explain your reasoning in comments.

Exception Chaining (Cause Tracking)

Key Points
• When one exception leads to another, you can chain them to preserve the root cause.
• Allows wrapping lower-level exceptions with meaningful higher-level ones.
• All Throwable classes support the cause mechanism.

Code Snippet

```
try {
    readConfigFile();
} catch (IOException e) {
    throw new ConfigurationException("Failed to load configuration", e);
}
```

Output Stack Trace Example:

```
ConfigurationException: Failed to load configuration
Caused by: java.io.FileNotFoundException: config.txt (No such file)
```

Exercise 05 – ExceptionChaining.java
Simulate reading a missing file and catch the IOException. Wrap it into a new `ConfigLoadException` using the constructor that accepts a cause.

Constructors Supporting Cause Chaining

Key Points

• All Throwable subclasses have a two-argument constructor for chaining.
• Syntax: `new Exception(String message, Throwable cause)`
• You can also set the cause later using `initCause(Throwable cause).`

Code Snippet

```
Exception e1 = new IOException("Low-level I/O issue");
Exception e2 = new CustomAppException("App failed", e1);
throw e2;

// OR
CustomAppException e = new CustomAppException("Failure");
e.initCause(e1);
throw e;
```

Exercise 06 – InitCauseDemo.java

Write code that throws a base exception (e.g., IOException) and then wraps it into a `BusinessException` using both `super(msg, cause)` and `initCause()` approaches. Observe the difference in the stack trace.

Full Example: Custom Exception with Chaining

Key Points
• Shows how custom exceptions preserve low-level causes for better debugging.

Code Snippet

```
class DataAccessException extends Exception {
    DataAccessException(String msg, Throwable cause) {
        super(msg, cause);
    }
}

void fetchData() throws DataAccessException {
    try {
        Files.readString(Path.of("data.txt"));
    } catch (IOException e) {
        throw new DataAccessException("Error reading data file", e);
    }
}
```

Output:

```
DataAccessException: Error reading data file
Caused by: java.io.IOException: data.txt not found
```

Exercise 07 – DataAccessChain.java
Simulate a database fetch operation that triggers an IOException. Wrap it into a custom `DataAccessException` and print the full stack trace to observe chaining.

Nested Try-Catch and Chaining

Key Points

• Use exception chaining in multi-layered applications:
• Data layer → low-level (e.g., SQLException)
• Service layer → wraps into business exceptions
• Controller layer → displays user-friendly message
• Example flow: SQLException → DataAccessException → ServiceException → UI Error Message

Code Snippet

```
try {
    fetchData();
} catch (DataAccessException e) {
    throw new ServiceException("Unable to fetch records", e);
}
```

Exercise 08 – MultiLayerChain.java
Build three layers: DataLayer (throws IOException), ServiceLayer (wraps into DataAccessException), and Controller (wraps into ServiceException). Print all causes in the topmost catch block.

Best Practices for Custom and Chained Exceptions

Key Points

• Create exceptions only when they add clarity or represent a new abstraction.
• Always include meaningful messages.
• Prefer chaining over suppressing original causes.
• Use checked exceptions for recoverable errors; unchecked for programming faults.
• Log exceptions with full stack traces before rethrowing or wrapping.

Exercise 09 – ExceptionBestPractices.java

Refactor a codebase that catches generic Exception. Replace it with meaningful custom exceptions, add cause chaining, and improve logging messages.

Key Takeaways

Key Points

• Custom exceptions make business logic more meaningful and self-documenting.
• Use chaining (`new Exception(msg, cause)`) to preserve the root cause.
• Checked exceptions force handling; unchecked simplify flow.
• A clear exception hierarchy improves debugging and separation of concerns.
• Never lose the original cause — always chain when rethrowing.

Module – Best Practices for Exception Propagation and Resource Management

Introduction

Key Points

• Exception handling and propagation determine how errors move through an application.
• Proper design ensures that errors are:
• Handled at the right level,
• Logged and wrapped appropriately, and
• Do not leak system resources (files, DB connections, sockets, etc.).
• This module focuses on propagation, wrapping, and safe cleanup.

Exercise 01 – ExceptionIntro.java

Write a short program that throws and catches an exception. Print the stack trace to understand how exception flow works.

What Is Exception Propagation?

Key Points

• If an exception is not caught in a method, it propagates upward to the caller.
• It continues until a matching catch block is found.
• If none is found, the program terminates with a stack trace.

Code Snippet

```
void methodA() { methodB(); }
void methodB() { methodC(); }
void methodC() { throw new RuntimeException("Error!"); }

methodA(); // propagates through call stack
```

Stack Trace Example:

```
Exception in thread "main" java.lang.RuntimeException: Error!
    at methodC(...)
    at methodB(...)
    at methodA(...)
```

Exercise 02 – PropagationDemo.java

Create three methods calling each other. Throw an exception in the innermost one and observe how it propagates through the call stack.

Deciding Where to Handle Exceptions

Key Points
• Handle exceptions as close as possible to where meaningful action can be taken.
• Let unresolvable exceptions bubble up to higher layers.
• Layered strategy:
• Low-level: Wrap technical issues (IOException, SQLException).
• Service-level: Translate into domain exceptions (DataAccessException).
• UI-level: Log and display user-friendly messages.
Example Flow: SQLException → DataAccessException → ServiceException → Display error

Exercise 03 – LayeredHandling.java
Simulate a multi-layered structure (Data → Service → Controller). Trigger an SQLException in the data layer and handle it meaningfully at the top layer.

When to Catch, When to Propagate

Scenario	Action
You can recover or retry	Catch and handle locally
You need to log or wrap	Catch, log, and rethrow
Caller can handle better	Let it propagate
Error is unrecoverable	Let JVM terminate or wrap in unchecked exception

Exercise 04 – CatchOrPropagate.java
Write two methods: one handles an exception locally (with retry) and another propagates it upward. Observe the difference in output.

Wrapping and Propagating Exceptions

Key Points

• Wrap low-level exceptions in higher-level ones with clear messages.
• Use constructors `(String message, Throwable cause)` to retain root cause.

Code Snippet

```
try {
    readDatabase();
} catch (SQLException e) {
    throw new DataAccessException("Failed to fetch records", e);
}
```

Output:

```
DataAccessException: Failed to fetch records
Caused by: java.sql.SQLException: Connection timeout
```

Exercise 05 – WrapAndPropagate.java

Create a method that catches an IOException and wraps it into a custom DataAccessException using a cause-chaining constructor.

Avoid Swallowing Exceptions

Key Points
• Never catch an exception and ignore it silently.
• Always log, rethrow, or handle meaningfully.

Bad Example:

```
try {
    performOperation();
} catch (Exception e) {
    // Ignored — dangerous!
}
```

Good Example:

```
try {
    performOperation();
} catch (Exception e) {
    log.error("Operation failed", e);
    throw e;
}
```

Exercise 06 – ExceptionSwallowing.java
Write two methods — one that swallows an exception and another that logs and rethrows it. Observe which gives better debugging output.

Logging and Rethrowing

Key Points
• Log exceptions once per layer — avoid duplicate logs.
• Use logging frameworks (e.g., SLF4J, Log4j) for consistency.
• Include relevant context (IDs, filenames, etc.) in messages.

Code Snippet

```
try {
    saveData();
} catch (IOException e) {
    logger.error("Failed to save report: {}", reportId, e);
    throw new ReportSaveException("Error saving report", e);
}
```

Exercise 07 – LogAndRethrow.java
Use System.err or a logger to record contextual details about an exception (like filename). Then rethrow it wrapped in a new exception.

Resource Management Basics

Key Points
• Resources like files, streams, and DB connections must be closed properly.
• Unreleased resources can cause leaks or exhaustion.
• Two approaches:

1. try–finally (older approach)
2. try-with-resources (modern, preferred)

Exercise 08 – ResourceLeakDemo.java
Open a FileReader without closing it. Use a memory or handle monitor (like VisualVM or logs) to see potential leaks.

Using try-with-resources

Key Points
• Introduced in Java 7 for automatic cleanup.
• Works with any class implementing AutoCloseable.
• Closes all resources even when an exception occurs.

Code Snippet

```
try (BufferedReader br = new BufferedReader(new FileReader("data.txt"))) {
    System.out.println(br.readLine());
} catch (IOException e) {
    System.out.println("File read failed: " + e.getMessage());
}
```

Benefits:
✓ No need for finally
✓ Cleaner and safer code

Exercise 09 – TryWithResourcesPractice.java
Convert your file-handling code from try–finally to try-with-resources. Confirm that the resource closes automatically even on exception.

Handling Exceptions in Resource Closing

Key Points

• If both the try block and close() throw exceptions, the close exception is suppressed.
• Access suppressed exceptions via `Throwable.getSuppressed()`.

Code Snippet

```
try (FaultyResource fr = new FaultyResource()) {
    throw new Exception("Main failure");
} catch (Exception e) {
    for (Throwable t : e.getSuppressed()) {
        System.out.println("Suppressed: " + t);
    }
}
```

Exercise 10 – SuppressedInClose.java

Create a class implementing AutoCloseable that throws an exception in close(). Inside try, throw another exception. Catch and print suppressed exceptions.

Common Anti-Patterns

Anti-Pattern	Why It's Bad
Empty catch block	Hides real issues
Overly broad catch (Exception)	Masks root cause
Logging & rethrowing same exception repeatedly	Duplicates stack trace
Manual resource close without finally	Risk of leaks
Using exceptions for flow control	Poor design and performance hit

Exercise 11 – AntiPatternFix.java
Identify and correct these anti-patterns in a small application. Replace each with best-practice code.

Best Practices Summary

For Propagation
• Catch only where you can handle meaningfully.
• Preserve the root cause with `(msg, cause)`.
• Avoid unnecessary wrapping or generic catches.
• Use custom exceptions for clarity at business boundaries.
For Resource Management
• Prefer try-with-resources over manual closing.
• Implement AutoCloseable for custom classes.
• Handle or log exceptions thrown during close operations.
• Release connections and files immediately after use.

Exercise 12 – ExceptionResourceBestPractices.java
Refactor an existing try–catch–finally codebase to follow these best practices. Add contextual logging and use AutoCloseable.

Key Takeaways

Key Points
• Handle exceptions at the right layer — based on ownership and context.
• Preserve and chain original causes — never suppress them.
• Log with context but avoid redundancy.
• try-with-resources ensures safe, automatic cleanup.
• Good propagation and cleanup improve reliability and maintainability.